

# SpecMamba: Accelerating Mamba Inference on FPGA with Speculative Decoding

Linfeng Zhong<sup>1,3\*</sup>, Songqiang Xu<sup>1,4\*</sup>, Huifeng Wen<sup>1,4</sup>, Tong Xie<sup>1,2</sup>, Qingyu Guo<sup>2</sup>, Yuan Wang<sup>2,5</sup>, Meng Li<sup>1,2,5†</sup>

<sup>1</sup>Institute for Artificial Intelligence, Peking University, Beijing, China

<sup>2</sup>School of Integrated Circuits, Peking University, Beijing, China

<sup>3</sup>School of Electronic and Computer Engineering, Peking University, Shenzhen, China

<sup>4</sup>School of Software and Microelectronics, Peking University, Beijing, China

<sup>5</sup>Beijing Advanced Innovation Center for Integrated Circuits, Beijing, China

**Abstract**—The growing demand for efficient long-sequence modeling on edge devices has propelled widespread adoption of State Space Models (SSMs) like Mamba, due to their superior computational efficiency and scalability. As its autoregressive generation process remains memory-bound, speculative decoding has been proposed that incorporates draft model generation and target model verification. However, directly applying speculative decoding to SSMs faces three key challenges: (1) hidden state backtracking difficulties, (2) tree-based parallel verification incompatibility, and (3) hardware workload mismatch. To address these challenges, we propose SpecMamba, the first FPGA-based accelerator for Mamba with speculative decoding, which features system, algorithm, and hardware co-design. At the system level, we present a memory-aware hybrid backtracking strategy to coordinate both models. At the algorithm level, we propose first-in-first-out (FIFO)-based tree verification with tiling to minimize memory access. At the hardware level, we customize a dataflow that computes linear layers in parallel and SSM layers in series to enable maximal overlapping. Implemented on AMD FPGA platforms (VHK158 and VCK190), SpecMamba achieves a  $2.27\times$  speedup over GPU baselines and a  $2.85\times$  improvement compared to prior FPGA solutions, while demonstrating  $5.41\times$  and  $1.26\times$  higher energy efficiency, respectively.

**Index Terms**—Mamba, tree-based speculative decoding, FPGA accelerator, dataflow architecture

## I. INTRODUCTION

State Space Models (SSMs), like Mamba [1], [2], have emerged as a competitive alternative to Transformer-based large language models (LLMs) [3], [4]. With linear computational complexity, Mamba has exhibited superior demonstrated superior efficiency [2] for long sequence modeling. Consequently, Mamba has undergone extensive development and refinement for various edge application scenarios, such as computer vision [5]–[7], multi-modal [8]–[10], medical application [11]–[14], and point cloud processing [15]–[17].

Despite Mamba’s impressive efficiency gains, its autoregressive generation process remains memory-bound, leading to suboptimal computational utilization and thus limited

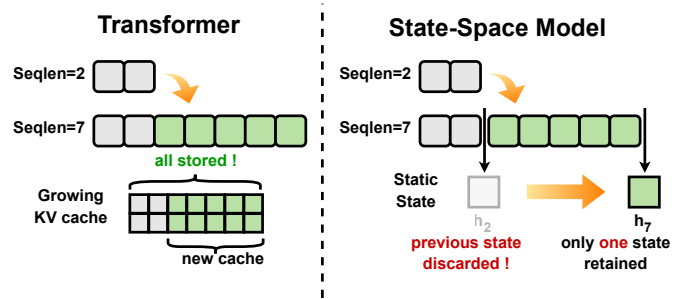


Fig. 1: Backtracking during speculative decoding. Transformers store all previous information in KV cache, while SSMs update and discard previous tokens, hindering backtracking.

throughput. To mitigate this bandwidth constraint, speculative decoding has been proposed as a promising solution [18], [19], effectively increasing computational density and alleviating bandwidth constraints without compromising generation quality. This technique employs a two-stage framework: (1) a smaller **draft model** generates potential token sequences autoregressively, while (2) a larger **target model** verifies these candidate tokens in parallel. Candidate tokens can be further organized into tree structures [20] to improve the acceptance rate of the draft tokens and the overall speedup ratio. However, unlike Transformer-based models that leverage key-value cache (KV cache) to store and select previous tokens, Mamba models dynamically compress historical token information into a fixed-size hidden state, impeding backtracking during speculative decoding, as depicted in Fig. 1. Therefore, current research [21], [22] focuses primarily on (1) efficient verification of multiple candidate tokens within a single forward pass and (2) reliable state recovery to backtrack to the last valid hidden state after token rejection.

Despite promising, effectively accelerating Mamba with speculative decoding on edge devices, such as FPGAs, remains an open challenge. [21], [22] primarily target GPU platforms and would incur prohibitive overhead if directly deployed on FPGAs. [23], [24] have explored Mamba acceleration in edge devices using conventional techniques such as quantization and operator optimization. However, none of them supports spec-

This work was supported in part by NSFC under Grant 62495102 and Grant 92464104, Grant 62341407, in part by the National Key Research and Development Program under Grant 2024YFB4505004, in part by Beijing Municipal Science and Technology Program under Grant Z241100004224015, and in part by 111 Project under Grant B18001.

\*Equal contribution. †Corresponding author.

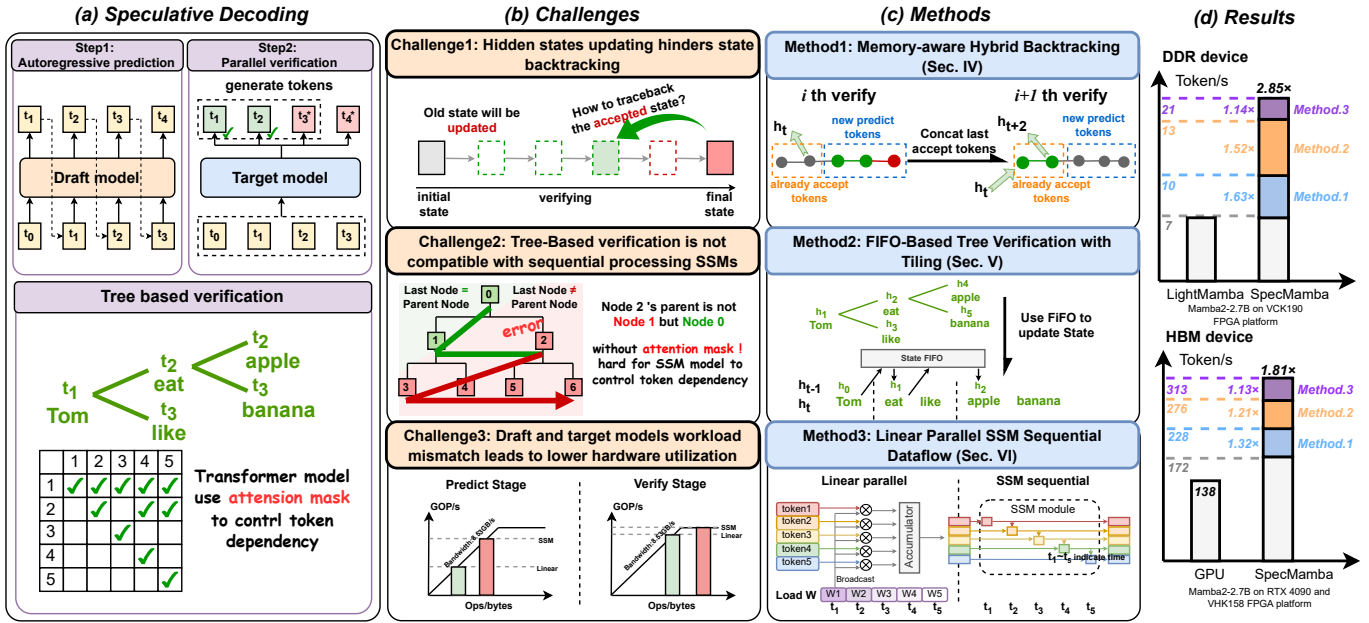


Fig. 2: Overview of *SpecMamba*. (a) Dataflow of speculative decoding. (b) Challenges of adapting speculative decoding for Mamba inference on FPGA. (c) Methods on *SpecMamba* from Sec. IV to Sec. VI. (d) Results on DDR and HBM devices.

ulative decoding. Applying speculative decoding to FPGA-accelerated Mamba presents three fundamental challenges, as illustrated in Fig. 2 (b): (1) **Hidden states updating hinders state backtracking.** Mamba discards previous hidden states after each update, complicating state recovery when draft tokens are rejected. (2): **Tree-based parallel verification is not compatible with sequential processing.** SSMs' sequential token processing precludes parallel path verification for tree-based drafts via attention masks [20] as in Transformers. (3) **Draft and target models workload mismatch leads to lower hardware utilization.** The autoregressive draft model is memory-bound while the parallel verification of target model is compute-intensive. Making both work efficiently on the same hardware without wasting resources is crucial.

To address the above challenges, we propose *SpecMamba*, the first FPGA-based accelerator that supports efficient Mamba inference with speculative decoding. As shown in Fig. 2 (c), At the system level, we present ① **Memory-aware hybrid backtracking**, which combines off-chip state storage for draft models with on-chip activation caching for target models. At the algorithm level, we propose ② **First-in-first-out (FIFO)-based tree verification with tiling** that enables tree-based speculative decoding in SSMs by exploiting inter-token dependencies for full-tree verification in a single pass. At the hardware level, we employ ③ **Linear Parallel SSM Sequential Dataflow**, where we schedule linear layer computation in parallel and SSM computations in series to facilitate overlapping. These optimizations achieve significant throughput improvements, as shown in Fig. 2 (d). Our main contributions can be summarized as follows:

- We present the first framework that accelerates Mamba inference with speculative decoding on FPGAs. Our

framework overcomes the incompatibilities between SSMs' sequential processing and speculative decoding's parallel verification requirements, enabling efficient high-speed inference on edge devices.

- We develop a cross-level optimization approach that integrates: (i) a memory-aware hybrid backtracking strategy, (ii) a novel FIFO-based tree verification with tiling algorithm, and (iii) a linear parallel SSM sequential dataflow.
- Our implementation on AMD platforms (VHK158 with HBM and VCK190 with DDR) achieves a 2.27x speedup and 5.41x higher energy efficiency compared to GPU baselines. Compared to prior FPGA-based works, *SpecMamba* demonstrates improvements of 2.85x in execution speed and 1.26x in energy efficiency.

## II. BACKGROUND AND RELATED WORK

### A. Mamba

The architecture of Mamba is shown in Fig. 3. Each Mamba block consists of two linear projection layers (i.e., an input projection and an output projection), a 1-dimensional convolution layer, a residual connection, an SSM layer for sequence modeling, and two RMSNorm layers. In each Mamba block, the input sequence  $X_l$  is processed to produce intermediate activations,  $\Delta, X, B, C$ . The SSM layer in Eq. 1 and Eq. 2 defines a linear mapping from an input signal  $X_t$  to output signal  $Y_t$  through a hidden state  $h_t$  in each timestep  $t$ :

$$h_t = \bar{A} \otimes h_{t-1} + \bar{B} \otimes X_t \quad (1)$$

$$Y_t = C \times h_{t-1} + D \otimes X_t \quad (2)$$

where  $\otimes$  denotes element-wise multiplication. A typical configuration (e.g., Mamba-2-2.7B with  $h = 80$ ,  $p = 64$ ,

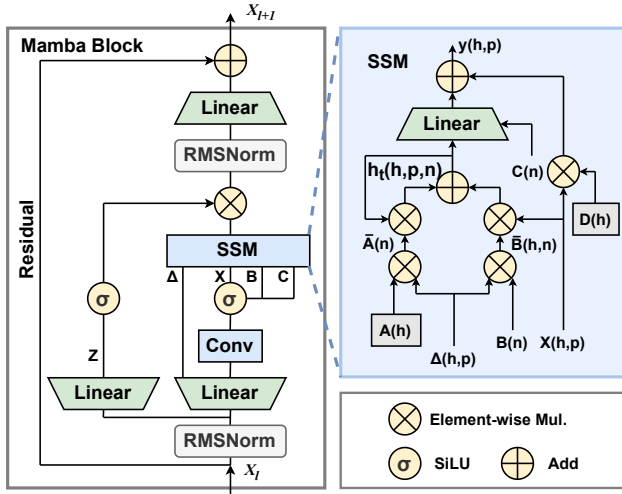


Fig. 3: The model architecture of Mamba2 and the detailed computation graph of the SSM layer.

$n = 128$ ) reveals that the hidden state  $h_t \in \mathbb{R}^{h \times p \times n}$  may incur over 1 GB memory overhead during parallel verification, thousands of times greater than activations such as  $A \in \mathbb{R}^h$ ,  $B \in \mathbb{R}^n$  and  $X \in \mathbb{R}^{h \times p}$ .

During SSM processing, the input  $X$  undergoes a sequence of transformations. Specifically,  $\Delta$  is involved in element-wise multiplications with parameters  $A$  and  $B$  separately, generating  $\bar{A}$  and  $\bar{B}$ .  $\bar{A}$  is then multiplied with the hidden state  $h_{t-1}$  and  $\bar{B}$  is element-wise multiplied with input  $X$ , respectively. The outcomes of these operations are then added to obtain an updated hidden state  $h_t$ . Finally,  $h_t$  is multiplied by  $C$ , followed by a linear transformation. The result is then added to the element-wise product of the input  $X$  and parameter  $D$ , generating the final output  $Y$ .

### B. Speculative decoding

Speculative decoding leverages smaller language models, referred to as draft models, to generate draft tokens, and adopts the original larger model, named target models, to verify draft tokens in parallel. When a token fails verification, the next iteration of draft generation and parallel verification rolls back from the last accepted draft tokens. To improve the acceptance rate, recent approaches adopt tree-based verification [25]–[29] rather than sequence-based verification [18], [19], which construct and verify a tree composed of draft tokens, as illustrated in Fig. 2 (a). By simultaneously considering more diverse speculation candidates (instead of just one as in sequence-based approaches), tree-based approaches can significantly boost the number of tokens generated in a single forward pass.

As depicted in Fig. 2 (a), while Transformer-based models use attention masks to restrict tokens (e.g.,  $h_5$  (“banana”)) to attend only their parent nodes (e.g.,  $h_1$  (“Tom”) and  $h_2$  (“eats”)) in the tree structure, Mamba models lack an analogous mechanism for selective token attention.

To address this, Snakes and Ladders [21] introduced two approaches, the first recovers states by recomputing from

TABLE I: Comparison between different LLM Accelerators.

|                  | Mamba Compatibility | Hardware Utilization | Decode Speedup | Energy Efficiency |
|------------------|---------------------|----------------------|----------------|-------------------|
| [30]–[32]        | ×                   | High                 | Middle         | High              |
| LightMamba [24]  | ✓                   | Low                  | Middle         | High              |
| MARCA [23]       | ✓                   | High                 | Low            | High              |
| Snakes [21]      | ✓                   | High                 | High           | Low               |
| SpecMamba (Ours) | ✓                   | High                 | High           | High              |

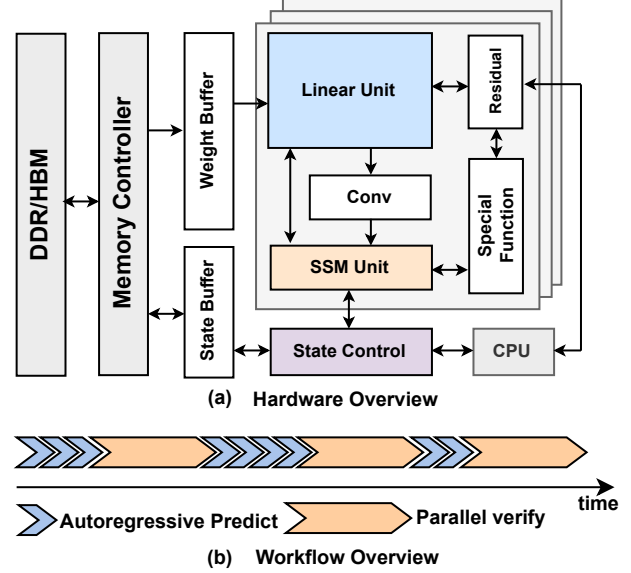


Fig. 4: SpecMamba overview.

augmented input sequences, and the second avoids intensive linear computation by maintaining intermediate activations. [22] preserved hidden states for both models, restoring them through selective recomputation during inference. However, these methods are designed for GPUs and require extensive tensor transfer, leading to unacceptable communication and memory overhead in bandwidth-limited FPGA platforms.

### C. LLM-related Accelerators

As listed in Table I, while traditional techniques such as quantization, sparsification, and operator fusion have demonstrated promising performance for accelerating Transformer-based models [30]–[33], they can not be directly applied to Mamba due to its complex computation graph and distinct activation distribution. MARCA [23] introduced reconfigurable operators to reconcile the incompatibility between matrix multiplication and element-wise operations, and LightMamba [24] focuses on improving decode-phase throughput with rotation-based quantization. However, the bandwidth bottleneck during autoregressive decoding remains unaddressed, leading to severe underutilization of FPGA computation resources.

Driven by these limitations, we propose SpecMamba, which integrates speculative decoding into an efficient architecture to alleviate memory bandwidth bottlenecks in FPGA-based Mamba inference. By leveraging idle FPGA compute capacity to parallelize draft generation and verification, SpecMamba accelerates the decoding phase and transforms underutilized

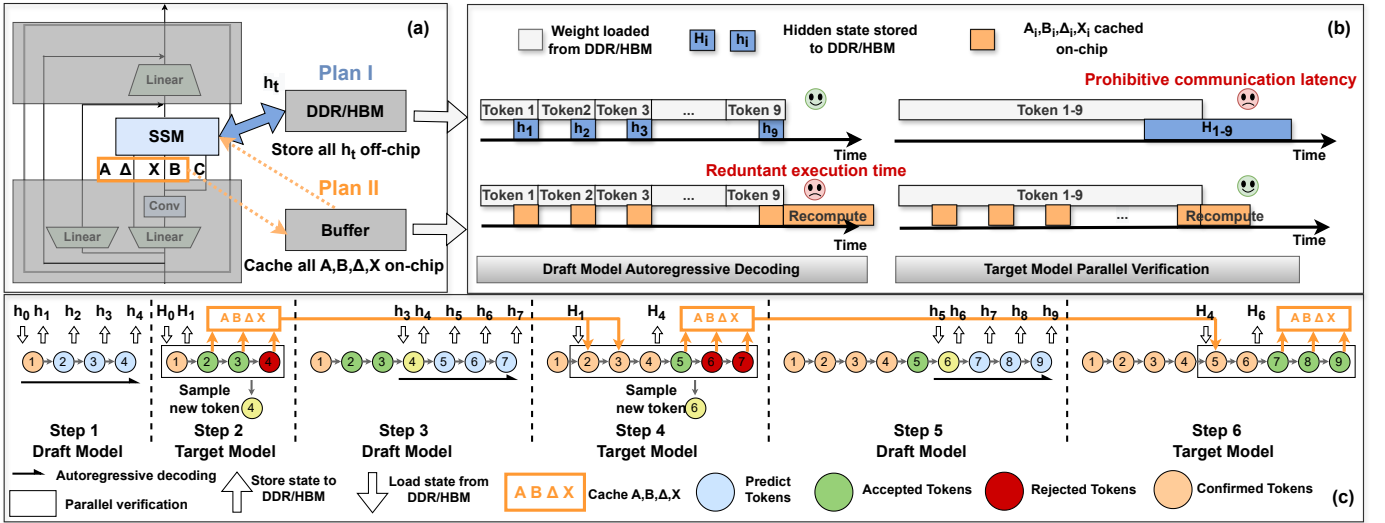


Fig. 5: Memory-aware hybrid backtracking. (a) Existing techniques for backtracking. Plan I stores all hidden states off-chip, while Plan II caches lightweight intermediate activations on chip and recomputes hidden states. (b) Tradeoff of Plan I and II for Draft Model and Target Model. For the draft model, Plan II incurs redundant recomputation time. For the target model, Plan I encounters non-overlappable communication overhead. (c) Hybrid backtracking strategy during speculative decoding.

resources into performance gains. Notably, speculative decoding is orthogonal to existing optimization techniques such as quantization, sparsification, and operator fusion, and thus can be jointly applied to further enhance inference efficiency.

### III. ARCHITECTURE OVERVIEW

In this section, we present the overview of *SpecMamba* as in Fig. 4 (a), which asynchronously executes draft and target models. This system features a hierarchical memory structure: model parameters are stored in off-chip DDR/HBM while intermediate states and weight tiles are cached within on-chip buffers. The design comprises the following components.

- Linear Unit: highly parallelized multiplication-and-accumulation (MAC) arrays for matrix operations.
- SSM Unit (SSMU): fully unrolled, pipelined element-wise multiplication units (EMUs) for SSM computations.
- State Controller: dynamically manages SSM states, issuing memory requests to store and retrieve states.
- Memory Controller: schedules data transfers between FPGA and off-chip memory.
- Special Function Unit (SFU): accelerates nonlinear functions and feeds results to the SSMU.
- Convolution (Conv) Unit: executes 1D convolutions.
- Residual Unit: computes residual connections and propagates outputs to subsequent layers or the CPU.

This unified architecture supports two modes: (1) autoregressive decoding (draft model), dynamically adjusting cycles for variable tree-based computation paths, and (2) parallel verification (target model), validating token trees with single weight loading execution (Fig. 4 (b)). This design balances resource efficiency, parallel execution, and adaptive scheduling for speculative decoding.

### IV. MEMORY-AWARE HYBRID BACKTRACKING

#### A. Challenge

While Mamba achieves memory efficiency by compressing previous tokens into a constant-sized hidden state, this design complicates backtracking in speculative decoding. When draft tokens are rejected, the generation process must roll back to the last accepted token, which is unpredictable before validation. Since Mamba's casual updates discard prior hidden states, such backtracking becomes difficult.

As shown in Fig. 5 (a), two main approaches exist to address this. **Plan I**: storing hidden states of all draft tokens [21], yet incurring substantial memory overhead. **Plan II**: caching lightweight activations ( $A, B, \Delta, X$ ) to recompute hidden states [22], which reduces memory overhead but introduces additional computation. Given the distinct workload of draft generation and parallel verification, balancing memory and computation is critical to optimizing FPGA deployments.

#### B. Observation

Mamba's hidden state generation involves linear projection and SSM computations. Linear layer weights are typically larger than hidden states [2], which requires tiled off-chip loading per execution. SSM operations incur minimal memory access. Fig. 5 (b) contrasts Plan I (store all states) and Plan II (recompute states) for draft and target models. For draft model autoregressive decoding with one weight load per token, Plan I enables hidden state loading overlapping with weight transfers, eliminating latency overhead. Plan II introduces recomputation penalties during hidden state regeneration. Conversely, target model parallel verification executes linear layers once but updates hidden states  $H_t$  via SSMs sequentially. Here, multiple hidden state storage of Plan I creates prohibitive communication latency, while hidden state recomputation in



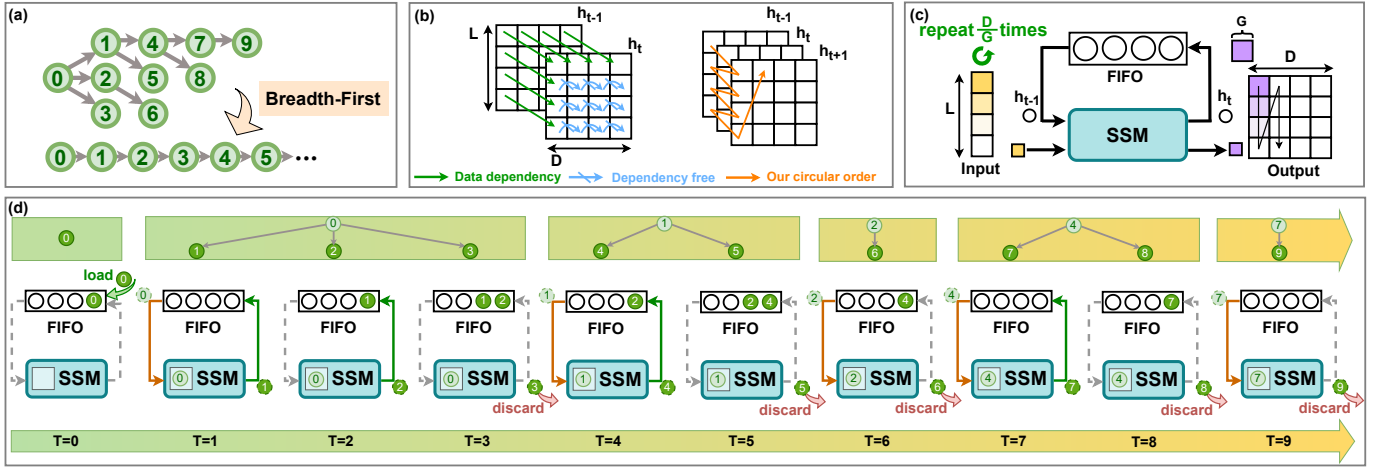


Fig. 6: FIFO-based tree verification with tiling. (a) Candidate tokens are structured into trees and flattened into sequences via breadth-first traversal. (b) Hidden states have inter-node dependencies, while intra-node tiles are independent, enabling the tiled computation workflow across nodes. (c) Hidden states are managed through an on-chip FIFO that tracks dependencies between tokens. (d) The execution of the tree node hidden state storage and computation is managed by FIFO.

Plan II optimizes throughput. This analysis demonstrates the critical trade-off: Plan I benefits draft models through memory-transfer overlapping, but incurs prohibitive storage overhead for target models. Plan II optimizes target model verification throughput, yet sacrifices draft generation efficiency due to recomputation costs.

### C. Approach

Based on these insights, we present a memory-aware hybrid hidden state backtracking strategy in Fig. 5 (c) that combines off-chip state storage for draft models with on-chip activation caching for target models. In step 1, the draft model generates tokens  $t_1 \sim t_4$ , storing hidden states  $h_1 \sim h_4$  off-chip. In step 2, the target model stores hidden state  $H_1$  off-chip while caching activations  $A, B, \Delta, X$  on-chip, then rejects  $t_4$  and resamples. In step 3, the draft model resumes from  $h_3$  to generate tokens  $t_4 \sim t_7$ , storing  $h_4 \sim h_7$ . In step 4, the target model recomputes  $H_2 \sim H_4$  using cached activations, with  $H_4$  stored off-chip to enable incremental recomputation in step 6. This execution pattern persists throughout subsequent steps, optimizing backtracking latency through strategic balancing of computational and memory costs.

## V. FIFO-BASED TREE VERIFICATION WITH TILING

### A. Challenge

A critical limitation of SSMs is their lack of mechanisms to enforce parent-only token attention. This creates a fundamental conflict in tree-structured draft verification: while SSMs inherently require sequential state dependencies, verifying flattened tree drafts (Fig. 6 (a)) disrupts causal structure. These non-monotonic state dependencies force all the intermediate hidden states to persist in either on-chip BRAM or off-chip HBM/DDR, as leaf nodes depend on them. The former incurs prohibitive on-chip memory overhead, while the latter introduces costly data transfer.

### B. Observation

Sec. II-A identifies prohibitive storage costs from retaining all draft tree hidden states. We observe that breadth-first traversal permits state eviction once the leaf nodes' verification is complete. For instance, *node 1* in Fig. 6 (a) is retained only until its child nodes (*node 4* and *node 5*) complete verification, after which it is discarded. By maintaining only required active nodes, this strategy can eliminate full-tree storage.

However, storing the entire hidden state exceeds on-chip buffer capacity, which motivates tensor size reduction. Eq. 1 reveals that  $h_t$  is computed from  $h_{t-1}$  via element-wise multiplication, yielding a key structural property as in Fig. 6 (b): while  $h_t$  tensors exhibit inter-token data dependencies, they have no intra-token dependencies. This enables decomposition of hidden states into tiles, replacing full-tensor storage with single-tile buffering while preserving causal integrity.

### C. Approach

To reduce memory overhead, we propose a hardware-friendly FIFO-based Tree Verification with Tiling. This algorithm employs breadth-first traversal and tile-wise processing. As illustrated in Fig. 6 (c), nodes are managed via an on-chip FIFO that tracks inter-token dependencies, with each node iterated  $D/G$  times, where  $D$  denotes hidden state dimension and  $G$  denotes tile size.

Fig. 6 (d) depicts execution for the tree in Fig. 6 (a). At timestep  $T = 0$ , the root node  $h_0$  is loaded into the FIFO and SSM module. The SSM computes  $h_1, h_2$ , and  $h_3$  at  $T = 1, 2, 3$ , respectively, with  $h_1$  and  $h_2$  forwarded to FIFO for leaf-node processing. Non-branching nodes  $h_3$  are discarded. Parent transitions trigger FIFO pops that retrieve nodes for computation in the SSM module (e.g.,  $T = 4, 6, 7, 9$ ), while cached states are reused during timesteps with unchanged parent nodes (e.g.,  $T = 2, 3, 5, 8$ ). Newly generated states are retained in the FIFO only for unresolved branches (e.g., *node*

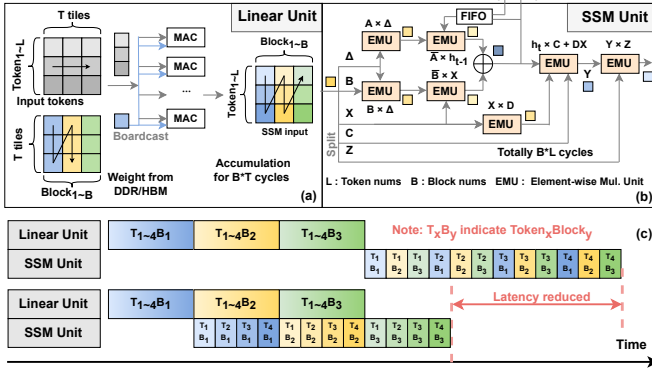


Fig. 7: Linear parallel SSM sequential dataflow. (a) Linear unit loads weight tiles with peak bandwidth and broadcasts them for parallel MAC operations across all tokens. (b) SSM unit unrolls multiple operators in parallel using EMUs and performs pipelined execution sequentially along the token dimension. (c) Sequential SSM execution and architecturally co-designed parallelism enable overlap with linear computation.

1, 2, 4, 7), while terminal nodes are discarded immediately (e.g., node 3, 5, 6, 8, 9).

The hidden states are partitioned into hardware-configurable tiles as in Fig. 6 (b), with each traversal iteration processing positionally aligned tiles across all nodes. Full hidden state computation is completed in  $D/G$  iterations. Combining node eviction with tiled execution, this approach only requires on-chip FIFO storage for at most  $N/2 \times G$ , where  $N$  denotes node count, avoiding bulk off-chip transfers. Overlapping tiled iterations can further minimize the latency overhead.

## VI. LINEAR PARALLEL SSM SEQUENTIAL DATAFLOW

### A. Challenge

Given that the draft model and target model work alternately and share a similar computation pipeline, a unified dataflow, rather than isolating them, is preferred to prevent hardware underutilization. However, their divergent computational demands pose challenges. The target model is compute-intensive, requiring tiled operations constrained by on-chip memory that increase SSM latency during multi-token verification. The draft model is memory-bound during sequential autoregressive token generation. Therefore, how to efficiently allocate DSP resources and schedule dataflow to simultaneously accommodate compute parallelism, memory bandwidth constraints, and overlap opportunity, remains a key design challenge.

### B. Observation

Hardware design for Mamba comprises two primary stages: linear layers and SSM layers, with the two following observations. (1) Linear layers are memory-bound, with latency dominated by off-chip weight access. Parallelism must align with memory bandwidth to prevent compute unit stalls. SSM layers, involving compute-bound element-wise operations on activations, require minimal off-chip traffic and can maximize parallelism within DSP constraints. (2) Furthermore,

their data dependencies differ: linear layers exhibit no inter-token dependence but require multi-tile input aggregation per output block (Fig. 7 (a)), while SSM layers show inter-token dependencies with no intra-token constraints (Fig. 6 (b)). This distinction enables overlap of linear and SSM computations through strategic scheduling, thus maximizing throughput.

### C. Approach

Building on these insights, we design a dataflow that parallelizes linear layers across tokens while processing SSMs sequentially. This supports both parallel verification for target models and autoregressive generation for small models, as shown in Fig. 7.

For linear units in Fig. 7 (a),  $L$  tokens are processed in parallel during verification. The weights are divided into  $B$  blocks along the output channel, and each block is further divided into  $T$  tiles along the input channel. The system maximizes its bandwidth utilization to load one tile from DDR/HBM and broadcasts it to multiple MAC units for matrix multiplication of all  $L$  tokens. After  $T$  cycles, the output of the first block is obtained, serving as input to the SSM Unit. The full linear unit computation is completed in  $B \times T$  cycles.

As shown in Fig. 7 (b), the SSM unit comprises a variety of operators, all of which are fully unrolled and organized for pipelined execution. This unrolling is achieved through EMUs, which enable efficient parallel expansion of these operators. Once the first block of input from the Linear unit (containing  $L$  tokens) becomes available, the SSM Unit can immediately begin pipelined computation. Unlike linear units, the SSM unit consumes only one tile of an individual token per cycle. Thus, it takes  $L$  cycles to process one block and  $L \times B$  cycles to complete the entire SSM computation.

Fig. 7 (c) demonstrates how we exploit inter-token dependencies assuming no intra-token constraints. By sequentially executing the same tile computation across different tokens in the SSM Unit and designing the parallelism such that the maximum  $L$  approximates  $T$ , we achieve full overlap between linear and SSM computations during parallel token processing. This design significantly reduces the system's overall latency.

## VII. EXPERIMENTS

### A. Evaluation Setup

We evaluate the performance of SpecMamba on AMD VHK158 for HBM scenario and AMD VCK190 for DDR scenario. Our design is compared with several prior works, including LightMamba [24] on FPGA and Snakes and Ladders [21] on NVIDIA RTX 4090 GPU to highlight the performance gains in terms of throughput, energy efficiency, and hardware utilization. The hardware specifications are detailed in Table II. To emulate an edge device where resources are strictly constrained, we deliberately excluded the use and performance evaluation of the dedicated AI Engine available on the VCK190 platform. SpecMamba is implemented using Vitis HLS and Vivado Design FLOW on both VHK158 and VCK190. Fig. 8 shows the layout of our implementation on VHK158 FPGA, and Table III presents its resource usage.

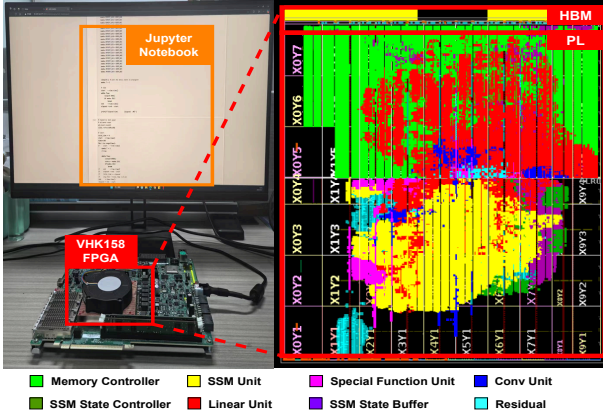


Fig. 8: SpecMamba implementation layout on VHK158.

TABLE II: Hardware Specification

|                      | NVIDIA<br>4090 GPU | AMD<br>VCK190 FPGA | AMD<br>VHK158 FPGA |
|----------------------|--------------------|--------------------|--------------------|
| <b>Compute units</b> | 512 tensor cores   | 1968 DSPs          | 7392 DSPs          |
| <b>Frequency</b>     | 2235 MHz           | 400 MHz            | 250 MHz            |
| <b>FP16 PCP</b>      | 330 TOPS           | 2.09 TOPS          | 4.91 TOPS          |
| <b>Memory</b>        | 24 GB              | 8 GB               | 32 & 32 GB         |
| <b>Bandwidth</b>     | 1008 GB/s          | 12 GB/s            | 819 & 51 GB/s      |

The throughput is measured using the PYNQ framework and power consumption with the Xilinx’s BEAM tool on-board. We quantize all models to INT4 precision following [24] and implement tree-based speculative decoding as proposed in [25]. The temperature for draft model prediction is set to 1.0, and the prediction length is set to 16 by default. Following [21], [24], we set the batch size to 1.

**Models and Datasets.** We use Mamba2-2.7B as the target model, and Mamba2-130M, 370M and 780M as draft models. We evaluate the throughput and energy efficiency on five widely adopted datasets: MT-Bench [34], PIQA [35], GSM-8K [36], Alpaca [37], and HumanEval [38], which are representative of question answering, mathematical reasoning, and code generation tasks.

## B. Evaluation Results

**Throughput and Energy Efficiency.** Fig. 9 presents the throughput (Tokens/s) and energy efficiency (Tokens/J) comparisons with these previous works under different draft model sizes: implementation on GPU without speculative decoding (Naive GPU), implementation on GPU with speculative decoding (Snakes) [21], and FPGA implementation without speculative decoding (LightMamba) [24]. SpecMamba consistently outperforms both GPU baselines across all three draft model sizes: 130M, 370M, and 780M. Mamba2-370M delivers the best overall performance, with average throughput improvements of  $3.12\times$ ,  $2.11\times$ , and  $1.17\times$ , and energy efficiency enhancements of  $1.26\times$ ,  $5.41\times$ , and  $2.73\times$ , over LightMamba, Naive GPU, and Snakes, respectively. Compared to the 370M draft model, the 130M model delivers lower

TABLE III: Resource Usage of SpecMamba on VHK158

| Component                   | LUT(k)     | FF(k)       | DSP         | BRAM        | URAM       |
|-----------------------------|------------|-------------|-------------|-------------|------------|
| <b>Memory Controller</b>    | 331.0      | 488.0       | 0           | 276         | 61         |
| <b>SSM Unit</b>             | 253.5      | 382.1       | 1068        | 56          | 12         |
| <b>SPU</b>                  | 35.1       | 53.6        | 193         | 163         | 12         |
| <b>Conv Unit</b>            | 17.3       | 41.5        | 295         | 132         | 9          |
| <b>SSM State Controller</b> | 31.5       | 49.2        | 0           | 56          | 12         |
| <b>Linear Unit</b>          | 254.0      | 403.4       | 3537        | 142         | 30         |
| <b>SSM State Buffer</b>     | 17.6       | 26.9        | 0           | 415         | 231        |
| <b>Residual</b>             | 34.9       | 53.3        | 0           | 294         | 61         |
| <b>Total</b>                | <b>975</b> | <b>1498</b> | <b>5093</b> | <b>1534</b> | <b>428</b> |

TABLE IV: Comparison with Prior Art Works

|                   | HBM Scenario |                            | DDR Scenario |                            |
|-------------------|--------------|----------------------------|--------------|----------------------------|
|                   | LightMamba   | Ours                       | LightMamba   | Ours                       |
| <b>Platform</b>   | U280 FPGA    | VHK158 FPGA                | VCK190 FPGA  | VCK190 FPGA                |
| <b>Frequency</b>  | 200MHz       | 250MHz                     | 400MHz       | 400MHz                     |
| <b>Bandwidth</b>  | 460GB/s      | 819GB/s                    | 12GB/s       | 12GB/s                     |
| <b>Model</b>      | Mamba2-2.7B  | Mamba2-2.7B<br>Mamba2-370M | Mamba2-2.7B  | Mamba2-2.7B<br>Mamba2-370M |
| <b>LUT</b>        | 297k(22.7%)  | 975k(56.6%)                | 107k(11.9%)  | 536k(59.6%)                |
| <b>FF</b>         | 394k         | 1498k                      | 130k         | 643k                       |
| <b>BRAM</b>       | 912          | 1534                       | 912          | 1223                       |
| <b>DSP</b>        | 1164(12.8%)  | 5093(68.8%)                | 228(11.5%)   | 1106(56.2%)                |
| <b>URAM</b>       | 61           | 428                        | 61           | 141                        |
| <b>Thoughtput</b> | 93           | 313.4                      | 7.21         | 20.6                       |

speedup, primarily due to its reduced acceptance rate, which limits the effectiveness of speculative execution. We also observe a performance drop when further increasing the model size from 370M to 780M. Although larger draft models tend to improve acceptance rates, they also introduce significantly higher computational overhead during decoding, which leads to reduced throughput and increased energy consumption.

We compare SpecMamba with the prior-art FPGA accelerator, LightMamba [24], across two hardware settings: HBM-FPGA and DDR-FPGA in Table IV. Both works employ Mamba2-2.7B as the target model, while SpecMamba integrates Mamba2-370M as the draft model. For the HBM scenario, SpecMamba achieves a practical throughput of 313.4 tokens/s on the VHK158 FPGA, which outperforms LightMamba on the U280 FPGA by  $3.37\times$  on average. For the DDR scenario, SpecMamba achieves a throughput of 20.6 tokens/s on the VCK190 FPGA, which is  $2.85\times$  higher than LightMamba. SpecMamba consistently outperforms prior works in both high-bandwidth and bandwidth-limited conditions.

TABLE V: Average tokens accepted per decoding step using Mamba2-2.7B as target and Mamba2-370M as draft model.

| Draft Structure | Dataset   | Prediction Lengths |      |      |      |      |      |
|-----------------|-----------|--------------------|------|------|------|------|------|
|                 |           | 6                  | 8    | 10   | 12   | 14   | 16   |
| <b>Sequence</b> | MT-Bench  | 1.78               | 1.97 | 2.24 | 2.39 | 2.58 | 2.71 |
|                 | GSM-8K    | 2.12               | 2.33 | 2.86 | 2.95 | 3.14 | 3.17 |
|                 | HumanEval | 2.05               | 2.28 | 2.68 | 2.89 | 3.04 | 3.09 |
| <b>Tree</b>     | MT-Bench  | 3.11               | 3.63 | 3.92 | 4.16 | 4.77 | 4.91 |
|                 | GSM-8K    | 3.82               | 4.41 | 4.81 | 5.03 | 5.94 | 5.98 |
|                 | HumanEval | 3.69               | 4.27 | 4.69 | 4.92 | 5.76 | 5.82 |

**Acceptance Rate.** In Table V, we also provide the ac-

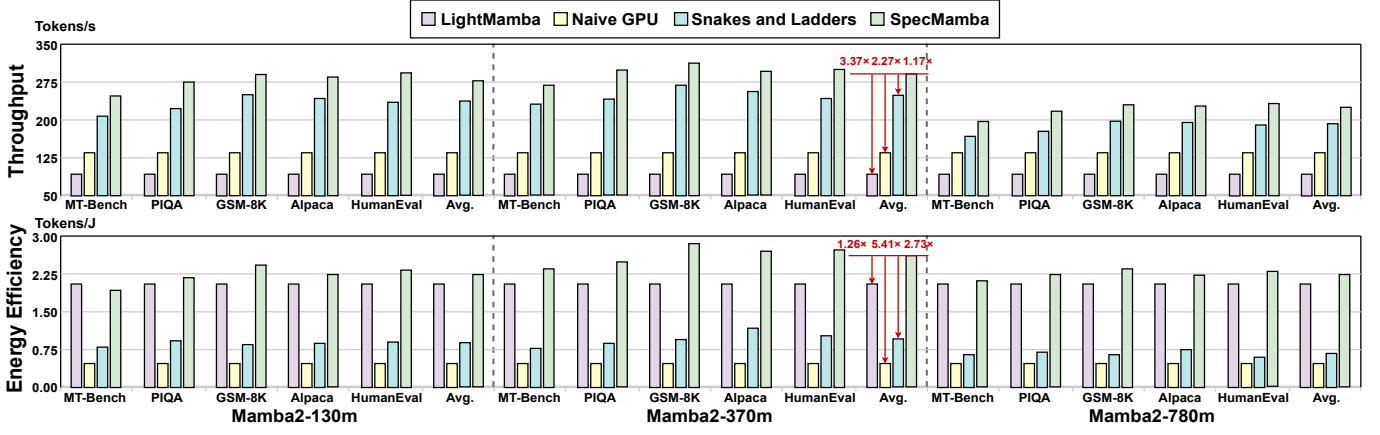


Fig. 9: Comparison of throughput and energy efficiency improvement of SpecMamba to LightMamba, Naive GPU, and Snakes and Ladders across three different draft model sizes.

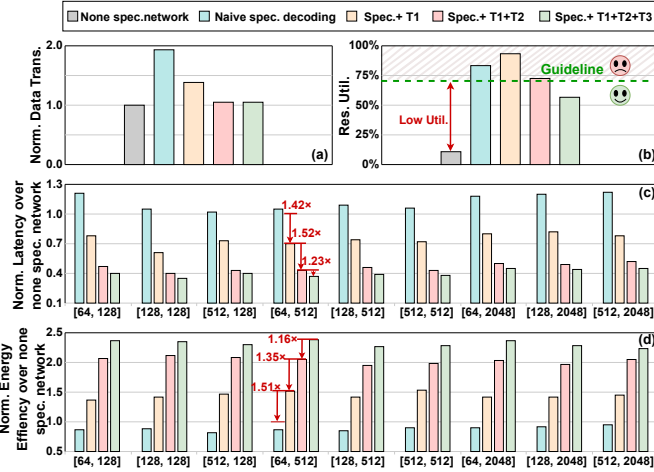


Fig. 10: Ablation study: (a) Normalized data transmission. (b) Resource utilization. (c) Normalized latency over None spec. network. (d) Normalized energy efficiency over None spec. network. T1: Memory-aware hybrid backtracking, T2: FIFO-based tree verification with tiling, T3: Linear parallel SSM sequential dataflow.

ceptance rates under different prediction lengths. Across all settings, our tree-based speculative decoding consistently achieves a higher acceptance rate compared to the sequence-based approach across all prediction lengths. Moreover, we also observe that the acceptance rate increases with prediction lengths, indicating more efficient draft token utilization in longer sequences.

**Ablation Study.** We conduct an ablation study on SpecMamba on the VCK190 platform to evaluate the impact of three different techniques on data transmission, resource consumption, latency, and energy efficiency in Fig. 10.

Fig. 10 (a) shows the effect of our methods on data transmission reduction. The Naive Spec. Decoding results in the highest transmission cost, as it stores all intermediate hidden states off-chip, leading to significant data movement. In contrast, our proposed techniques progressively reduce

the transmission volume, bringing it closer to that of the normalized None Spec. Network baseline.

In Fig. 10 (b), we illustrate the chip resource utilization of different methods, defined as the average utilization rate of on-chip LUT and DSP resources. Note that a widely used guideline for FPGA resource utilization is 70%, above which may potentially affect timing and routing. The None spec. network suffers from bandwidth limitations, resulting in low utilization far below the 70% guideline. Naive spec. decoding and Spec. +T1 causes overly high utilization, which could potentially impact timing and routing. Incorporation of T2 and T3 reduces utilization to 57.9%, keeping it below the guideline and making the design more routing-friendly.

Fig. 10 (c) and (d) show the normalized latency and energy efficiency over the None spec. network using our techniques, across various prefill and decoding lengths. For example, for a prefill length of 64 and a decode stage of 512, T1, T2, T3 reduce the latency by 1.42 $\times$ , 1.52 $\times$ , 1.23 $\times$ , and improve the energy efficiency by 1.51 $\times$ , 1.35 $\times$ , 1.16 $\times$ , respectively, demonstrating a gradual reduction in latency.

## VIII. CONCLUSION

This work presents SpecMamba, the first FPGA-accelerated framework enabling efficient Mamba inference with speculative decoding for edge deployment. We propose a cross-layer optimization framework to balance the sequential processing of SSMs and parallel verification of speculative decoding: (1) a memory-aware hybrid backtracking strategy that coordinates both models to resolve hidden state recovery bottlenecks, (2) a FIFO-based tree verification scheme with a tiling algorithm that enables full candidate token verification in SSMs, and (3) a linear parallel and SSM sequential dataflow that achieves balancing of the model workload through hardware-software co-design. Implemented on AMD VHK158 and VCK190 FPGAs, SpecMamba achieves 2.27 $\times$  speedup over GPU baselines and 2.85 $\times$  over prior FPGA designs, with 5.41 $\times$  and 1.26 $\times$  higher energy efficiency, respectively. These results validate SSMs for high-throughput edge applications and establish a new co-design paradigm for sequential state-space architectures.



## REFERENCES

- [1] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv preprint arXiv:2312.00752*, 2023.
- [2] T. Dao and A. Gu, “Transformers are ssms: Generalized models and efficient algorithms through structured state space duality,” *arXiv preprint arXiv:2405.21060*, 2024.
- [3] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, “Llama: Open and efficient foundation language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2302.13971>
- [4] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin *et al.*, “Opt: Open pre-trained transformer language models,” *arXiv preprint arXiv:2205.01068*, 2022.
- [5] Y. Liu, Y. Tian, Y. Zhao, H. Yu, L. Xie, Y. Wang, Q. Ye, J. Jiao, and Y. Liu, “Vmamba: Visual state space model,” *Advances in neural information processing systems*, vol. 37, pp. 103 031–103 063, 2024.
- [6] C. Yang, Z. Chen, M. Espinosa, L. Ericsson, Z. Wang, J. Liu, and E. J. Crowley, “Plainmamba: Improving non-hierarchical mamba in visual recognition,” *arXiv preprint arXiv:2403.17695*, 2024.
- [7] L. Zhu, B. Liao, Q. Zhang, X. Wang, W. Liu, and X. Wang, “Vision mamba: Efficient visual representation learning with bidirectional state space model,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.09417>
- [8] Z. Wan, P. Zhang, Y. Wang, S. Yong, S. Stepputtis, K. Sycara, and Y. Xie, “Sigma: Siamese mamba network for multi-modal semantic segmentation,” in *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. IEEE, 2025, pp. 1734–1744.
- [9] H. Zhao, M. Zhang, W. Zhao, P. Ding, S. Huang, and D. Wang, “Cobra: Extending mamba to multi-modal large language model for efficient inference,” *arXiv preprint arXiv:2403.14520*, 2024.
- [10] W. Dong, H. Zhu, S. Lin, X. Luo, Y. Shen, X. Liu, J. Zhang, G. Guo, and B. Zhang, “Fusion-mamba for cross-modality object detection,” *arXiv preprint arXiv:2404.09146*, 2024.
- [11] Z. Wang, J.-Q. Zheng, Y. Zhang, G. Cui, and L. Li, “Mamba-unet: Unet-like pure visual mamba for medical image segmentation,” *arXiv preprint arXiv:2402.05079*, 2024.
- [12] Y. Yang, Z. Xing, L. Yu, C. Huang, H. Fu, and L. Zhu, “Vivim: A video vision mamba for medical video segmentation,” *arXiv preprint arXiv:2401.14168*, 2024.
- [13] T. Guo, Y. Wang, S. Shu, D. Chen, Z. Tang, C. Meng, and X. Bai, “Mambamorph: a mamba-based framework for medical mr-ct deformable registration,” *arXiv preprint arXiv:2401.13934*, 2024.
- [14] Z. Xing, T. Ye, Y. Yang, G. Liu, and L. Zhu, “Segmamba: Long-range sequential modeling mamba for 3d medical image segmentation,” in *International Conference on Medical Image Computing and Computer-Assisted Intervention*. Springer, 2024, pp. 578–588.
- [15] Y. Li, W. Yang, and B. Fei, “3dmambacomplete: Exploring structured state space model for point cloud completion,” *arXiv preprint arXiv:2404.07106*, 2024.
- [16] T. Zhang, H. Yuan, L. Qi, J. Zhang, Q. Zhou, S. Ji, S. Yan, and X. Li, “Point cloud mamba: Point cloud learning via state space model,” *arXiv preprint arXiv:2403.00762*, 2024.
- [17] D. Liang, X. Zhou, W. Xu, X. Zhu, Z. Zou, X. Ye, X. Tan, and X. Bai, “Pointmamba: A simple state space model for point cloud analysis,” *arXiv preprint arXiv:2402.10739*, 2024.
- [18] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 19 274–19 286.
- [19] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, “Accelerating large language model decoding with speculative sampling,” *arXiv preprint arXiv:2302.01318*, 2023.
- [20] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi *et al.*, “Specinfer: Accelerating large language model serving with tree-based speculative inference and verification,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 932–949.
- [21] Y. Wu, Y. Dukler, M. Trager, A. Achille, W. Xia, and S. Soatto, “Snakes and ladders: Accelerating state space model inference with speculative decoding,” 2024.
- [22] J. Wang, D. Paliotta, A. May, A. Rush, and T. Dao, “The mamba in the llama: Distilling and accelerating hybrid models,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 62 432–62 457, 2024.
- [23] J. Li, S. Huang, J. Xu, J. Liu, L. Ding, N. Xu, and G. Dai, “Marca: Mamba accelerator with reconfigurable architecture,” in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, 2024, pp. 1–9.
- [24] R. Wei, S. Xu, L. Zhong, Z. Yang, Q. Guo, Y. Wang, R. Wang, and M. Li, “Lightmamba: Efficient mamba acceleration on fpga with quantization and hardware co-design,” *arXiv preprint arXiv:2502.15260*, 2025.
- [25] J. Wang, Y. Su, J. Li, Q. Xia, Z. Ye, X. Duan, Z. Wang, and M. Zhang, “Opt-tree: Speculative decoding with adaptive draft tree structure,” *Transactions of the Association for Computational Linguistics*, vol. 13, pp. 188–199, 2025.
- [26] Y. Li, F. Wei, C. Zhang, and H. Zhang, “Eagle: Speculative sampling requires rethinking feature uncertainty,” *arXiv preprint arXiv:2401.15077*, 2024.
- [27] —, “Eagle-2: Faster inference of language models with dynamic draft trees,” *arXiv preprint arXiv:2406.16858*, 2024.
- [28] —, “Eagle-3: Scaling up inference acceleration of large language models via training-time test,” *arXiv preprint arXiv:2503.01840*, 2025.
- [29] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, “Medusa: Simple llm inference acceleration framework with multiple decoding heads,” *arXiv preprint arXiv:2401.10774*, 2024.
- [30] S. Zeng, J. Liu, G. Dai, X. Yang, T. Fu, H. Wang, W. Ma, H. Sun, S. Li, Z. Huang *et al.*, “Flightllm: Efficient large language model inference with a complete mapping flow on fpgas,” in *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2024, pp. 223–234.
- [31] J. Li, T. Li, G. Shen, D. Zhao, Q. Zhang, and Y. Zeng, “Pushing up to the limit of memory bandwidth and capacity utilization for efficient llm decoding on embedded fpga,” *arXiv preprint arXiv:2502.10659*, 2025.
- [32] M. Huang, A. Shen, K. Li, H. Peng, B. Li, Y. Su, and H. Yu, “Edgellm: A highly efficient cpu-fpga heterogeneous edge accelerator for large language models,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2025.
- [33] A. Moitra, A. Ghosh, S. Agarwal, A. Amarnath, K. Swaminathan, and P. Panda, “Meadow: Memory-efficient dataflow and data packing for low power edge llms,” *arXiv preprint arXiv:2503.11663*, 2025.
- [34] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing *et al.*, “Judging llm-as-a-judge with mt-bench and chatbot arena,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 46 595–46 623, 2023.
- [35] Y. Bisk, R. Zellers, J. Gao, Y. Choi *et al.*, “Piqa: Reasoning about physical commonsense in natural language,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 05, 2020, pp. 7432–7439.
- [36] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano *et al.*, “Training verifiers to solve math word problems, 2021,” URL <https://arxiv.org/abs/2110.14168>, vol. 9, 2021.
- [37] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, “Stanford alpaca: An instruction-following llama model,” 2023.
- [38] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.